Assignment 2\A2_P1_Code.py

```python
import os
import numpy as np
import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation, FFMpegWriter

# Initializing Variables
x_start = 0
x_end   = 1
y_start = -0.5
y_end   = 0.5
t_start = 0
t_end   = 0.001
nx      = 101 #number of grid points in x
ny      = 101 #number of grid points in y
CFL     = 0.5
R       = 287 #gas constant
gamma   = 1.4 #ratio of specific heats

# Calculating problem parameters
dx = (x_end-x_start)/(nx-1)
dy = (y_end-y_start)/(ny-1)
x  = np.linspace(x_start,x_end,nx) #establishing discretized x locations
y  = np.linspace(y_start,y_end,ny) #establishing discretized x locations

# Initializing Variable Matricies
u_n     = np.zeros((len(x), len(y)))  #Initializing current time grid point u vector
u_np1   = np.zeros((len(x), len(y)))  #Initializing next time grid point u vector
v_n     = np.zeros((len(x), len(y)))  #Initializing current time grid point v vector
v_np1   = np.zeros((len(x), len(y)))  #Initializing next time grid point v vector
rho_n   = np.zeros((len(x), len(y)))  #Initializing current time grid point density vector
rho_np1 = np.zeros((len(x), len(y)))  #Initializing next time grid point demsotu vector
p_n     = np.zeros((len(x), len(y)))  #Initializing current time grid point pressure vector
p_np1   = np.zeros((len(x), len(y)))  #Initializing next time grid point pressure
E_n     = np.zeros((len(x), len(y)))  #Initializing current time grid point rho vector
E_np1   = np.zeros((len(x), len(y)))  #Initializing next time grid point rho vector

# Initial Conditions
u0   = 2000 + u_n
v0   = v_n.copy()
rho0 = 1 + rho_n
p0   = 101325 + p_n
E0   = ((p0/(gamma-1)) + (0.5*rho0*(u0**2+v0**2)))/rho0

# Applying initial conditions to current time step matricies
u_n   = u0.copy()
v_n   = v0.copy()
rho_n = rho0.copy()
```

```
p_n   = p0.copy()
E_n   = E0.copy()

# Initializing time history matricies
u_history   = np.empty((nx,ny,0)) #Initializing a time history u vector to store information for animations
v_history   = np.empty((nx,ny,0)) #Initializing a time history v vector to store information for animations
rho_history = np.empty((nx,ny,0)) #Initializing a time history density vector to store information for animations
p_history   = np.empty((nx,ny,0)) #Initializing a time history pressure vector to store information for animations
E_history   = np.empty((nx,ny,0)) #Initializing a time history energy vector to store information for animations
M_history   = np.empty((nx,ny,0)) #Initializing a time history mach number vector to store information for animations
t_history   = []

# Creating time variables
a = np.sqrt((gamma*p_n)/rho_n)          #Calculating the speed of sound at every point in space
dt_x = (CFL*dx)/np.max(np.abs(u_n) + a) #Calculating the most restrictive dt I can use based on a constant CFL number and the maximum u in my domain
dt_y = (CFL*dy)/np.max(np.abs(v_n) + a) #Calculating the most restrictive dt I can use based on a constant CFL number and the maximum u in my domain
dt   = min([dt_x,dt_y])                 #Choosing the smallest dt from the minumum dt of x and y

t_current = 0

box_x_start_idx = np.argmin(np.abs(x - 0.45))
box_x_end_idx   = np.argmin(np.abs(x - 0.55))
box_y_start_idx = np.argmin(np.abs(y - (-0.05)))
box_y_end_idx   = np.argmin(np.abs(y - 0.05))

while t_current < t_end: #Loop through time until we have simulated up until the end time
    for j in range(ny): #Loop through y grid points (j==0, y==-0.5)
        for i in range(nx): #Loop through the x grid points (i==0, x==0)
            if (j == 0)or(j == ny-1)or(i==0):
              # Top, Bottom, and Inlet Boundary Conditions
              rho_np1[i,j] = 1
              u_np1[i,j]   = 2000
              v_np1[i,j]   = 0
              p_np1[i,j]   = 101325
              E_np1[i,j]   = ((p_np1[i,j]/(gamma-1)) + (0.5*rho_np1[i,j]*(u_np1[i,j]**2+v_np1[i,j]**2)))/rho_np1[i,j]
            elif(i==nx-1):
              # Zero Gradient Boundary Condition for Outlet
              u_np1[i,j]   = u_np1[i-1,j]
              v_np1[i,j]   = v_np1[i-1,j]
              rho_np1[i,j] = rho_np1[i-1,j]
```

```
            p_np1[i,j]   = p_np1[i-1,j]
            E_np1[i,j]   = E_np1[i-1,j]

elif(i>box_x_start_idx)and(i<box_x_end_idx)and(j>box_y_start_idx)and(j<box_y_end_idx):
            # Nothing happens inside the box, in this region the points will always represent the initial condition and will skip the flow evolution update
            u_np1[i,j]   = u_n[i,j]
            v_np1[i,j]   = v_n[i,j]
            rho_np1[i,j] = rho_n[i,j]
            p_np1[i,j]   = p_n[i,j]
            E_np1[i,j]   = E_n[i,j]
         elif(i==box_x_start_idx)and(j>=box_y_start_idx)and(j<=box_y_end_idx):
            # Slip-wall conditions for the box left wall
            u_np1[i,j]   = -u_n[i-1,j]
            v_np1[i,j]   = v_n[i-1,j]
            rho_np1[i,j] = rho_n[i-1,j]
            p_np1[i,j]   = p_n[i-1,j]
            E_np1[i,j]   = E_n[i-1,j]
         elif(i==box_x_end_idx)and(j>=box_y_start_idx)and(j<=box_y_end_idx):
            # Slip-wall conditions for the box right wall
            u_np1[i,j]   = -u_n[i+1,j]
            v_np1[i,j]   = v_n[i+1,j]
            rho_np1[i,j] = rho_n[i+1,j]
            p_np1[i,j]   = p_n[i+1,j]
            E_np1[i,j]   = E_n[i+1,j]
         elif(j==box_y_start_idx)and(i>box_x_start_idx)and(i<box_x_end_idx):
            # Slip-wall conditions for the box bottom wall
            u_np1[i,j]   = u_n[i,j-1]
            v_np1[i,j]   = -v_n[i,j-1]
            rho_np1[i,j] = rho_n[i,j-1]
            p_np1[i,j]   = p_n[i,j-1]
            E_np1[i,j]   = E_n[i,j-1]
         elif(j==box_y_end_idx)and(i>box_x_start_idx)and(i<box_x_end_idx):
            # Slip-wall conditions for the box top wall
            u_np1[i,j]   = u_n[i,j+1]
            v_np1[i,j]   = -v_n[i,j+1]
            rho_np1[i,j] = rho_n[i,j+1]
            p_np1[i,j]   = p_n[i,j+1]
            E_np1[i,j]   = E_n[i,j+1]
         else:
           rho_np1[i,j] = (
                          0.25*(rho_n[i+1,j] + rho_n[i-1,j] + rho_n[i,j+1] + rho_n[i,j-1]) -
                          dt*(
                             ((rho_n[i+1,j]*u_n[i+1,j]-rho_n[i-1,j]*u_n[i-1,j])/(2*dx)) +
                             ((rho_n[i,j+1]*v_n[i,j+1]-rho_n[i,j-1]*v_n[i,j-1])/(2*dy))
                           )
                          )
            u_np1[i,j]   = (
                          0.25*(rho_n[i+1,j]*u_n[i+1,j] + rho_n[i-1,j]*u_n[i-1,j] + rho_n[i,j+1]*u_n[i,j+1] + rho_n[i,j-1]*u_n[i,j-1]) -
```

```
                                dt*(
                                   ((rho_n[i+1,j]*u_n[i+1,j]*u_n[i+1,j]-rho_n[i-1,j]*u_n[i-1,j]*u_n[i-1,j])/(2*dx)) +
                                   ((rho_n[i,j+1]*u_n[i,j+1]*v_n[i,j+1]-rho_n[i,j-1]*u_n[i,j-1]*v_n[i,j-1])/(2*dy)) +
                                   ((p_n[i+1,j]-p_n[i-1,j])/(2*dx))
                                   )
                                )/rho_np1[i,j]
            v_np1[i,j]   = (
                                0.25*(rho_n[i+1,j]*v_n[i+1,j] + rho_n[i-1,j]*v_n[i-1,j] + rho_n[i,j+1]*v_n[i,j+1] + rho_n[i,j-1]*v_n[i,j-1]) -
                                dt*(
                                   ((rho_n[i+1,j]*v_n[i+1,j]*u_n[i+1,j]-rho_n[i-1,j]*v_n[i-1,j]*u_n[i-1,j])/(2*dx)) +
                                   ((rho_n[i,j+1]*v_n[i,j+1]*v_n[i,j+1]-rho_n[i,j-1]*v_n[i,j-1]*v_n[i,j-1])/(2*dy)) +
                                   ((p_n[i,j+1]-p_n[i,j-1])/(2*dy))
                                   )
                                )/rho_np1[i,j]
            E_np1[i,j]   = (
                                0.25*(rho_n[i+1,j]*E_n[i+1,j] + rho_n[i-1,j]*E_n[i-1,j] + rho_n[i,j+1]*E_n[i,j+1] + rho_n[i,j-1]*E_n[i,j-1]) -
                                dt*(
                                   ((rho_n[i+1,j]*u_n[i+1,j]*E_n[i+1,j]-rho_n[i-1,j]*u_n[i-1,j]*E_n[i-1,j])/(2*dx)) +
                                   ((rho_n[i,j+1]*E_n[i,j+1]*v_n[i,j+1]-rho_n[i,j-1]*E_n[i,j-1]*v_n[i,j-1])/(2*dy)) +
                                   ((p_n[i+1,j]*u_n[i+1,j]-p_n[i-1,j]*u_n[i-1,j])/(2*dx)) +
                                   ((p_n[i,j+1]*v_n[i,j+1]-p_n[i,j-1]*v_n[i,j-1])/(2*dy))
                                )
                                ) / rho_np1[i,j]
            p_np1[i,j] = (gamma-1) * (rho_np1[i,j]*E_np1[i,j] - 0.5*rho_np1[i,j]*(u_np1[i,j]**2+v_np1[i,j]**2))

            if (rho_np1[i,j] <= 0) or (p_np1[i,j] <= 0) or \
             (not np.isfinite(rho_np1[i,j])) or (not np.isfinite(p_np1[i,j])):

             print("FIRST BAD FLUID POINT")
             print("i, j =", i, j)
             print("x, y =", x[i], y[j])
             print("rho_np1 =", rho_np1[i,j])
             print("u_np1 =", u_np1[i,j])
             print("v_np1 =", v_np1[i,j])
             print("E_np1 =", E_np1[i,j])
             print("p_np1 =", p_np1[i,j])
             print("dt =", dt)
             raise SystemExit
    #Updating the time
    t_current = t_current + dt

    #Updating the current time parameters
```

```
    u_n = u_np1.copy()
    v_n = v_np1.copy()
    rho_n = rho_np1.copy()
    p_n = p_np1.copy()
    E_n = E_np1.copy()

    a    = np.sqrt((gamma*p_n)/rho_n)       #Calculating the speed of sound at every point in space
    dt_x = (CFL*dx)/np.max(np.abs(u_n) + a) #Calculating the most restrictive dt I can use based on a constant CFL number and the maximum u in my domain
    dt_y = (CFL*dy)/np.max(np.abs(v_n) + a) #Calculating the most restrictive dt I can use based on a constant CFL number and the maximum u in my domain
    dt   = min([dt_x,dt_y])                 #Choosing the smallest dt from the minumum dt of x and y


    M_n = np.sqrt(u_n**2+v_n**2)/a

    #Keeping track of variables in history
    u_history   = np.dstack((u_history,u_n))
    v_history   = np.dstack((v_history,v_n))
    rho_history = np.dstack((rho_history,rho_n))
    p_history   = np.dstack((p_history,p_n))
    E_history   = np.dstack((E_history,E_n))
    M_history   = np.dstack((M_history,M_n))

    t_history.append(t_current)
# ============================================================
# CONTOURF ANIMATIONS
# ============================================================

def animate_contour(history, variable_name, units, cmap="viridis"):

    fig, ax = plt.subplots(figsize=(8,6))

    # Fixed contour limits for the full animation
    data_min = np.nanmin(history)
    data_max = np.nanmax(history)

    # Prevent contourf from failing if min == max
    if data_min == data_max:
        data_min = data_min - 1e-12
        data_max = data_max + 1e-12

    levels = np.linspace(data_min, data_max, 50)

    # Initial contour plot
    contour = ax.contourf(
        x,
        y,
        history[:,:,0].T,
```

```
        levels=levels,
        cmap=cmap
    )

    # Colorbar
    cbar = fig.colorbar(contour, ax=ax)
    cbar.set_label(f"{variable_name} [{units}]")

    # Axis formatting
    ax.set_xlabel("x [m]")
    ax.set_ylabel("y [m]")
    ax.set_xlim(x_start, x_end)
    ax.set_ylim(y_start, y_end)
    ax.set_title(
        f"{variable_name} at t = {t_history[0]:.6e} s"
    )


    def update(frame):

        ax.clear()

        ax.contourf(
            x,
            y,
            history[:,:,frame].T,
            levels=levels,
            cmap=cmap
        )

        ax.set_xlabel("x [m]")
        ax.set_ylabel("y [m]")
        ax.set_xlim(x_start, x_end)
        ax.set_ylim(y_start, y_end)

        ax.set_title(
            f"{variable_name} at t = {t_history[frame]:.6e} s"
        )


    animation = FuncAnimation(
        fig,
        update,
        frames=history.shape[2],
        interval=50,
        repeat=True,
        blit=False
    )

    return animation
```

```
275
276
277 # ============================================================
278 # SAVE CONTOURF ANIMATIONS AS MP4 VIDEOS
279 # ============================================================
280
281 import os
282 from matplotlib.animation import FuncAnimation, FFMpegWriter
283
284
285 def save_contour_video(history, variable_name, units, filename,
286                        cmap="turbo", fps=20):
287
288     print(f"Creating {variable_name} animation...")
289
290     fig, ax = plt.subplots(figsize=(8,6))
291
292     # Fixed contour limits over the entire simulation
293     data_min = np.nanmin(history)
294     data_max = np.nanmax(history)
295
296     # Prevent contourf error if the field is constant
297     if data_min == data_max:
298         data_min -= 1e-12
299         data_max += 1e-12
300
301     levels = np.linspace(data_min, data_max, 50)
302
303     # Initial contour
304     contour = ax.contourf(
305         x,
306         y,
307         history[:,:,0].T,
308         levels=levels,
309         cmap=cmap
310     )
311
312     # Colorbar
313     cbar = fig.colorbar(contour, ax=ax)
314     cbar.set_label(f"{variable_name} [{units}]")
315
316     ax.set_xlabel("x [m]")
317     ax.set_ylabel("y [m]")
318     ax.set_xlim(x_start, x_end)
319     ax.set_ylim(y_start, y_end)
320
321     title = ax.set_title(
322         f"{variable_name} at t = {t_history[0]:.6e} s"
323     )
324
```

```
325
326     def update(frame):
327
328         ax.clear()
329
330         ax.contourf(
331             x,
332             y,
333             history[:,:,frame].T,
334             levels=levels,
335             cmap=cmap
336         )
337
338         ax.set_xlabel("x [m]")
339         ax.set_ylabel("y [m]")
340         ax.set_xlim(x_start, x_end)
341         ax.set_ylim(y_start, y_end)
342
343         ax.set_title(
344             f"{variable_name} at t = {t_history[frame]:.6e} s"
345         )
346
347
348     animation = FuncAnimation(
349         fig,
350         update,
351         frames=history.shape[2],
352         interval=50,
353         blit=False
354     )
355
356     writer = FFMpegWriter(
357         fps=fps,
358         bitrate=2500
359     )
360
361     animation.save(
362         filename,
363         writer=writer,
364         dpi=150
365     )
366
367     # Close this figure before making the next video
368     plt.close(fig)
369
370     print(f"Saved: {os.path.abspath(filename)}")
371
372
373 # ============================================================
374 # SAVE EACH VIDEO ONE AT A TIME
```

```
# ============================================================

save_contour_video(
    u_history,
    "X Velocity",
    "m/s",
    "u_velocity.mp4"
)

save_contour_video(
    v_history,
    "Y Velocity",
    "m/s",
    "v_velocity.mp4"
)

save_contour_video(
    rho_history,
    "Density",
    "kg/m^3",
    "density.mp4"
)

save_contour_video(
    p_history,
    "Pressure",
    "Pa",
    "pressure.mp4"
)

save_contour_video(
    E_history,
    "Specific Total Energy",
    "J/kg",
    "energy.mp4"
)

save_contour_video(
    M_history,
    "Mach Number",
    "-",
    "mach_number.mp4"
)

print("All videos finished.")
```